

定制主题

定制主题

Ant Design 设计规范和技术上支持灵活的样式定制，以满足业务和品牌上多样化的视觉需求，包括但不限于全局样式（主色、圆角、边框）和指定组件的视觉定制。

自从 5.0 版本以来，我们提供了一套全新的定制主题方案。不同于 4.x 版本的 less 和 CSS 变量，有了 CSS-in-JS 的加持后，动态主题的能力也得到了加强，包括但不限于：

1. 支持动态切换主题；
2. 支持同时存在多个主题；
3. 支持针对某个/某些组件修改主题变量；
4. ...

配置主题

我们把影响主题的最小元素称为 **Design Token**。通过修改 Design Token，我们可以呈现出各种各样的主题或者组件。通过在 ConfigProvider 中传入 theme 属性，可以配置主题。

:::warning ConfigProvider 对 message.xxx、Modal.xxx、notification.xxx 等静态方法不会生效，原因是在这些方法中，antd 会通过 ReactDOM.render 动态创建新的 React 实体。其 context 与当前代码所在 context 并不相同，因而无法获取 context 信息。

当你需要 context 信息（例如 ConfigProvider 配置的内容）时，可以通过 Modal.useModal 方法返回 modal 实体以及 contextHolder 节点，将其插入到你需要获取 context 位置即可。也可通过 App 包裹组件 简化 useModal 等方法需要手动植入 contextHolder 的问题。 :::

修改主题变量

通过 theme 中的 token 属性，可以修改一些主题变量。部分主题变量会引起其他主题变量的变化，我们把这些主题变量称为 Seed Token。

使用预设算法

通过修改算法可以快速生成风格迥异的主题，我们默认提供三套预设算法，分别是：

- 默认算法 theme.defaultAlgorithm
- 暗色算法 theme.darkAlgorithm
- 紧凑算法 theme.compactAlgorithm

你可以通过 theme 中的 algorithm 属性来切换算法，并且支持配置多种算法，将会依次生效。

修改组件变量

除了整体的 Design Token，各个组件也会开放自己的 Component Token 来实现针对组件的样式定制能力，不同的组件之间不会相互影响。同样地，也可以通过这种方式来覆盖组件的其他 Design Token。

:::info{title=组件级别的主题算法} 默认情况下，所有组件变量都仅仅是覆盖，不会基于 Seed Token 计算派生变量。

在 >= 5.8.0 版本中，组件变量支持传入 algorithm 属性，可以开启派生计算或者传入其他算法。 :::

禁用动画

antd 默认内置了一些组件交互动效让企业级页面更加富有细节，在一些极端场景可能会影响页面交互性能，如需关闭动画可以 token 中的 motion 修改为 false：

进阶使用

零运行时 zeroRuntime {#zero-runtime}

自 6.0.0 起，我们提供了 zeroRuntime 模式来进一步提升应用性能。开启后，Ant Design 将不再在运行时生成组件样式，所以需要自行引入样式文件。

```
);
```

antd/dist/antd.css 包含了所有 antd 组件的样式，但是不会包含 hashed className。如果你希望引入更少的样式，或者因为修改了 prefix 等配置无法使用默认的样式，推荐使用 @ant-design/static-style-extract 来生成静态样式。

```
const cssText = extractStyle({
  includes: ['Button'], // 只包含 Button 组件的样式
});
```

```
fs.writeFileSync('/path/to/somewhere', cssText);
```

动态切换

在 v5 中，动态切换主题对用户来说是非常简单的，你可以在任何时候通过 ConfigProvider 的 theme 属性来动态切换主题，而不需要任何额外配置。

局部主题（嵌套主题）

可以嵌套使用 ConfigProvider 来实现局部主题的更换。在子主题中未被改变的 Design Token 将会继承父主题。

使用 Design Token

如果你希望使用当前主题下的 Design Token，我们提供了 useToken 这个 hook 来获取 Design Token。

静态消费（如 less）

当你需要非 React 生命周期消费 Token 变量时，可以通过静态方法 getDesignToken 将其导出：

```
const { getDesignToken } = theme;
```

```
const globalToken = getDesignToken();
```

getDesignToken 和 ConfigProvider 一样，支持传入 theme 属性，用于获取指定主题的 Design Token。

```
const { getDesignToken, useToken } = theme;
```

```
const config: ThemeConfig = {  
  token: {  
    colorPrimary: '#1890ff',  
  },  
};
```

```
// 通过静态方法获取
```

```
const globalToken = getDesignToken(config);
```

```
// 通过 hook 获取
const App = () => {
  const { token } = useToken();
  return null;
};

// 渲染示意
createRoot(document.getElementById('#app')).render(

,
);
```

如果需要将其应用到静态样式编译框架，如 less 可以通过 less-loader 注入：

```
{
  loader: "less-loader",
  options: {
    lessOptions: {
      modifyVars: mapToken,
    },
  },
}
```

兼容包提供了变量转换方法用于转成 v4 的 less 变量，如需使用[点击此处查看详情](#)。

调试主题

我们提供了帮助用户调试主题的工具：主题编辑器

你可以使用此工具自由地修改 Design Token，以达到你对主题的期望。

基本概念

在 Design Token 中我们提供了一套更加贴合设计的三层结构，将 Design Token 拆解为 Seed Token、Map Token 和 Alias Token 三部分。这三组 Token 并不是简单的分组，而是一个三层的派生关系，由 Seed Token 派生 Map Token，再由 Map Token 派生 Alias Token。在大部分情况下，使用 Seed Token 就可以满足定制主题的需要。但如果您需要更高层次的主题定制，您需要了解 antd 中 Design Token 的生命周期。

演变过程

图片说明：token

基础变量 (Seed Token)

Seed Token 意味着所有设计意图的起源。比如我们可以通过改变 colorPrimary 来改变主题色，antd 内部的算法会自动的根据 Seed Token 计算出对应的一系列颜色并应用：

```
const theme = {  
  token: {  
    colorPrimary: '#1890ff',  
  },  
};
```

梯度变量 (Map Token)

Map Token 是基于 Seed 派生的梯度变量。定制 Map Token 推荐通过 theme.algorithm 来实现，这样可以保证 Map Token 之间的梯度关系。也可以通过 theme.token 覆盖，用于单独修改一些 map token 的值。

```
const theme = {  
  token: {  
    colorPrimaryBg: '#e6f7ff',  
  },  
};
```

别名变量 (Alias Token)

Alias Token 用于批量控制某些共性组件的样式，基本上是 Map Token 别名，或者特殊处理过的 Map Token。

```
const theme = {  
  token: {  
    colorLink: '#1890ff',  
  },  
};
```

基本算法 (algorithm)

基本算法用于将 Seed Token 展开为 Map Token，比如由一个基本色算出一个梯度色板，或者由一个基本的圆角算出各种大小的圆角。算法可以单独使用，也可以任意地组合使用，比如可以将暗色算法和紧凑算法组合使用，得到一个暗色和紧凑相结合的主题。

```
const { darkAlgorithm, compactAlgorithm } = theme;
```

```
const theme = {  
  algorithm: [darkAlgorithm, compactAlgorithm],  
};
```

API

Theme

- 属性：token；说明：用于修改 Design Token；类型：AliasToken；默认值：-
- 属性：inherit；说明：继承上层 ConfigProvider 中配置的主题。；类型：boolean；默认值：true
- 属性：algorithm；说明：用于修改 Seed Token 到 Map Token 的算法；类型：(token: SeedToken) => MapToken | ((token: SeedToken) => MapToken)[]；默认值：defaultAlgorithm
- 属性：components；说明：用于修改各个组件的 Component Token 以及覆盖该组件消费的 Alias Token；类型：ComponentsConfig；默认值：-
- 属性：cssVar；说明：CSS 变量配置；类型：cssVar；默认值：-
- 属性：hashed；说明：将样式添加至 hash className 上；类型：boolean；默认值：true
- 属性：zeroRuntime；说明：开启零运行时模式，不会在运行时产生样式，需要手动引入 CSS 文件；类型：boolean；默认值：false；版本：6.0.0

ComponentsConfig

- 属性：Component (可以是任意 antd 组件名，如 Button)；说明：用于修改 Component Token 以及覆盖该组件消费的 Alias Token；类型：ComponentToken & AliasToken & { algorithm: boolean | (token: SeedToken) => MapToken | ((token: SeedToken) => MapToken)[]}；默认值：-

组件级别的 algorithm 默认为 false，此时组件 Token 仅仅会覆盖该组件使用的 token，不会进行派生计算。设置为 true 时会继承当前全局算法；也可以和全局的 algorithm 一样传入一个或多个算法，这将会针对该组件覆盖全局的算法。

cssVar {#css-var}

- 属性：prefix；说明：CSS 变量的前缀，默认与 ConfigProvider 上配置的 prefixCls 相同；类型：string；默认值：ant
- 属性：key；说明：当前主题的唯一识别 key，默认用 useId 填充；类型：string；默认值：useId in React 18

SeedToken

MapToken

继承所有 SeedToken 的属性

AliasToken

继承所有 SeedToken 和 MapToken 的属性

FAQ

为什么 theme 从 undefined 变为对象或者变为 undefined 时组件重新 mount 了？

在 ConfigProvider 中我们通过 DesignTokenContext 传递 context，theme 为 undefined 时不会套一层 Provider，所以从无到有或者从有到无时 React 的 VirtualDOM 结构变化，导致组件重新 mount。解决方法：将 undefined 替换为空对象 {} 即可。